

What is the output ?

```
m := map[string]int{"a": 1}
v, ok := m["b"]
fmt.Println(v, ok)

v2 := m["b"]
fmt.Println(v2 + 10)
```

```
a := make([]int, 3, 5)
a[0], a[1], a[2] = 1, 2, 3

b := append(a, 4)
b[0] = 100

fmt.Println(a)
fmt.Println(b)
```

```
m := map[string]int{"a": 1}
v, ok := m["b"]
fmt.Println(v, ok)

v2 := m["b"]
fmt.Println(v2 + 10)
```

```
funcs := make([]func(), 0, 3)
for i := 0; i < 3; i++ {
    funcs = append(funcs, func() { fmt.Println(i) })
}
for _, f := range funcs {
    f()
}
```

```
var ErrNotFound = errors.New("not found")
func find() error {
    return fmt.Errorf("lookup failed: %w", ErrNotFound)
}
func main() {
    err := find()
    fmt.Println(err)
    fmt.Println(err == ErrNotFound)
    fmt.Println(errors.Is(err, ErrNotFound))
}
```

```
defer func() {
    if r := recover(); r != nil {
        fmt.Println("Recovered in main:", r)
    }
}()
go func() {
    panic("goroutine panic!")
}()

time.Sleep(1 * time.Second)
fmt.Println("main finished")
```

```
defer fmt.Println("deferred 1")
defer func() {
    panic("panic in defer")
}()
defer fmt.Println("deferred 3")
fmt.Println("main body")
```

```
type Data struct {
    Values []int
}

func main() {
    d1 := Data{Values: []int{1}}
    d2 := Data{Values: []int{1}}
    fmt.Println(d1 == d2)
}
```

Good Luck 😊